SYSTEMS MANAGEMENT



Hierarchical Device Managers to manage Complex, Multi-Rack Systems

Avinash Natarajan, Senior Principal, Celestica

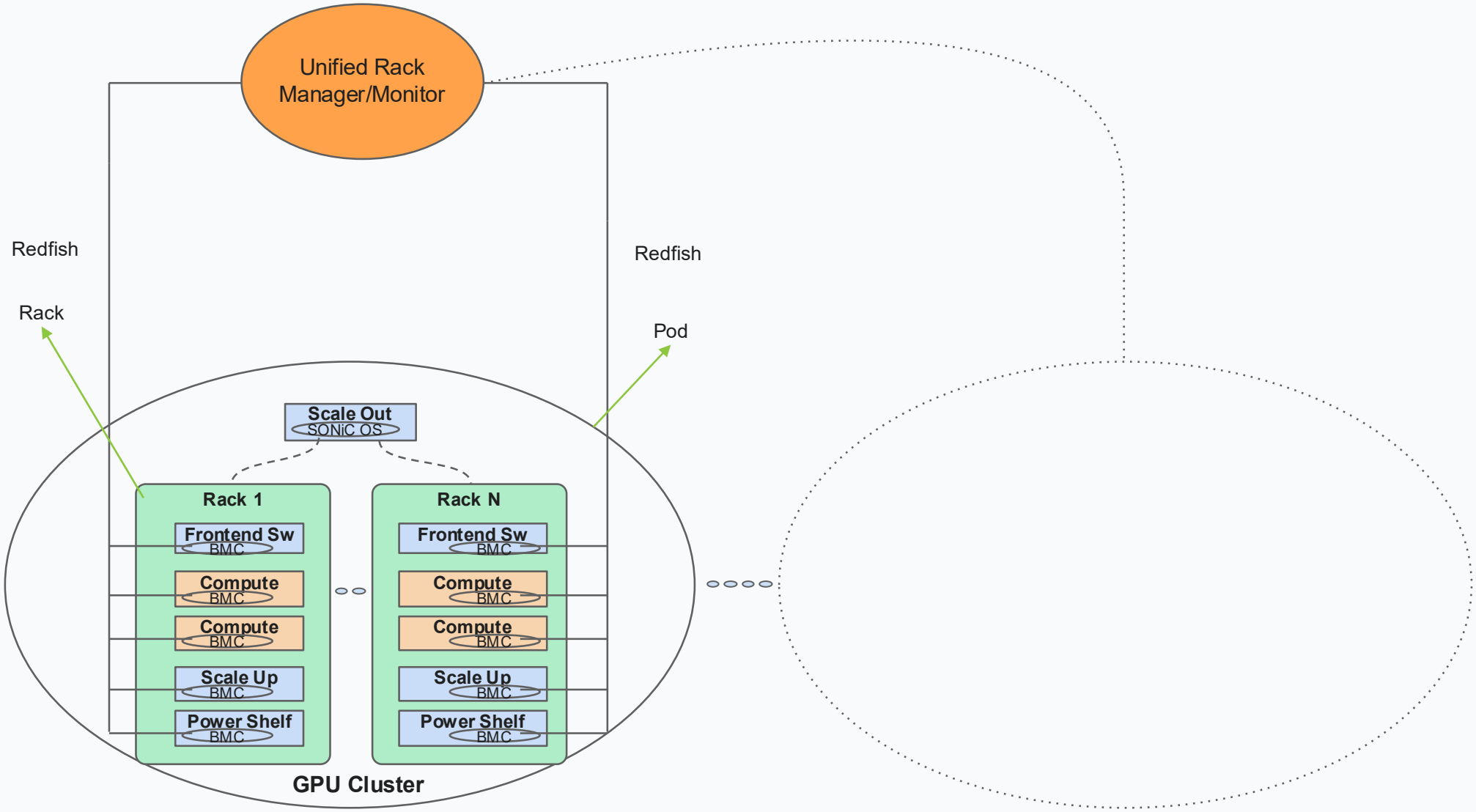
Navinsennivel Nallasivam, Staff, Celestica

OCP PLATINUM Member

Hierarchy in a Multi-Rack XPU Cluster

- Manageability Applications need a notion of Hierarchy while managing Multi-Rack XPU Clusters.
- Purpose of Hierarchy: Besides providing some structure/order, Initiating concurrent Operations on multiple identical elements is one utility (eg; upgrade all scale up switches)
- OpenRMC 1.3 specification defines a Hierarchy (Rack → Pod → Data Center), and in practice, this potentially lives in proprietary Manageability Applications
- There is value in
  - Standardizing how this Hierarchy is achieved, and Operations on top of it
  - Leaving the meaning of these Hierarchies open
- We will explore extending OpenRMC-DM to achieve the standardization part

Hierarchy in a Multi-Rack XPU Cluster



Managing Multi-Rack Infrastructure — What Are We Solving For?

The Rack Baseline

OpenRMC-DM today — at the rack level:

- One DM per rack manages multiple BMCs via Redfish
- Abstracts N BMCs into a single unified Redfish namespace
- The proxy aggregator pattern is established and works well

The gap — no standard above the rack:

- Modern deployments span multiple racks, pods, and data centers
- No standard exists for managing multiple DMs from a Northbound Application
- Each vendor builds proprietary multi-rack management

Northbound Burden Today

What a northbound application must implement in code today:

```
NorthBound Application
├── Maintain sessions to every Level-1 DM independently
├── Implement fan-out — issue every operation to N DMs
├── Correlate events & detect cross-rack patterns
├── Aggregate inventory, health, compliance
└── All of this grows linearly with rack count
```

The Result:

- NorthBound application complexity scales with deployment size
- Redfish — the standard that works for one rack — stops at the boundary

What We Are Proposing

Extend the rack-level proxy aggregator pattern upward.

We propose defining a standard hierarchy of nested DMs so that Redfish works uniformly from a single BMC to an entire data center.

OpenRMC-DM — A Rack-Level Redfish Aggregator

ARCHITECTURAL OVERVIEW

- **DM runs per rack** — either on a control card co-located with the BMCs it manages or on a server
- **Abstracts multiple BMCs** into a single unified Redfish namespace
- **Core DM Responsibilities:** Session management, event subscription, inventory aggregation, and task tracking
- **Demultiplexing:** The UUID created by the DM per aggregation source onboarded, is the demultiplexing key that identifies which BMC owns a resource

UUID NAMESPACING

Every BMC resource is namespaced with a unique device UUID to prevent collisions:

BMC Local:
`/redfish/v1/Systems/system`

Via DM:
`/redfish/v1/Systems/{deviceUUID}.system`

This is the proxy aggregator pattern.
DM already proves it works at the rack level.

Device Manager (DM) as a Rack-Level Proxy

ARCHITECTURE & INTERACTIVE FLOW

How the Device Manager acts as a Redfish API proxy for the entire rack:

- **Single Endpoint:** Northbound callers target one DM IP, completely hidden from individual internal BMC network addresses.
- **Redfish Mapping:** Demultiplexes incoming standard requests to target BMCs using registered UUID associations.
- **Response Rewrite:** Transparently transforms local BMC resource links into unique global endpoints.
- **Local Autonomy:** Each BMC remains unchanged, executing native firmware commands under `/redfish/v1/Systems/system`.

URI TRANSFORMATION PIPELINE

BMC Local URI (Natively on Server)

`/redfish/v1/Systems/system`

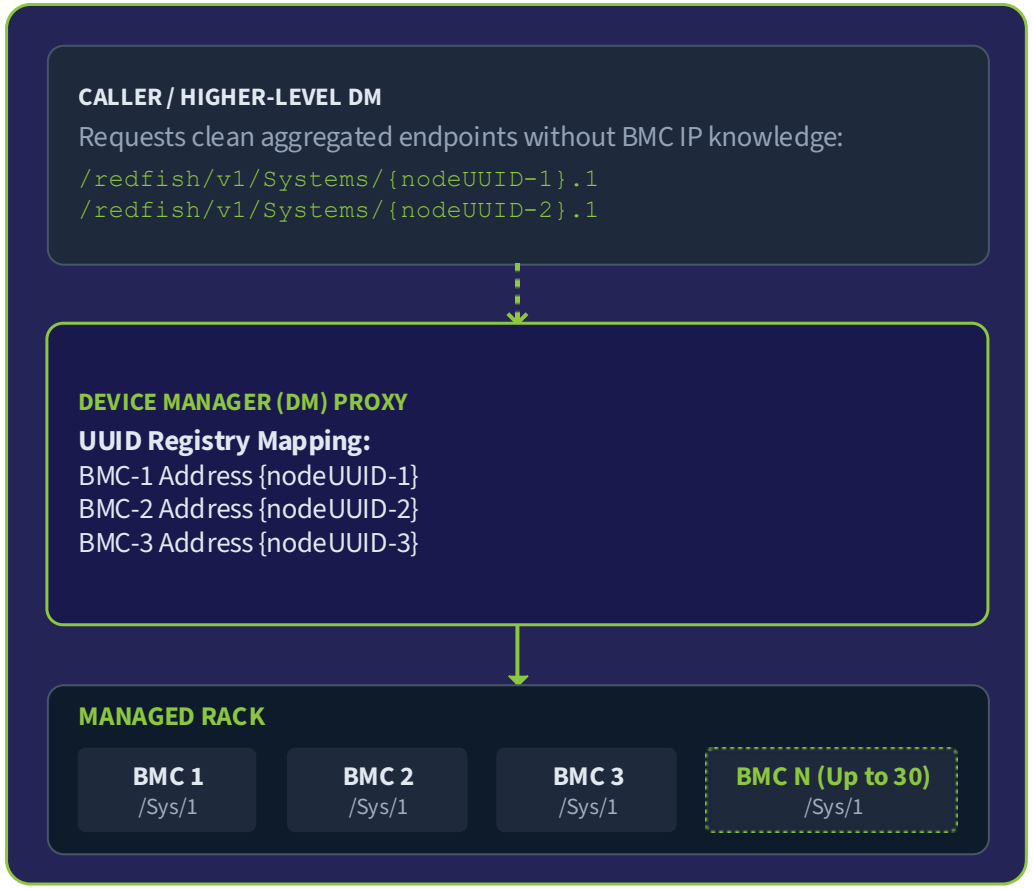
▼

Unified Namespace (Exposed via DM Proxy)

`/redfish/v1/Systems/{nodeUUID}.system`

`/redfish/v1/Chassis/{nodeUUID}.chassis`

`/redfish/v1/Managers/{nodeUUID}.bmc`



The Same Pattern, Applied Recursively

NESTED SYSTEM ARCHITECTURE

A Level-2 DM treats Level-1 DMs exactly as Level-1 DM treats BMCs:

- **Recursive Scaling:** Level-2 DM assigns a UUID to each Level-1 DM, namespacing all resources beneath it.
- **Identical Interface:** At every hierarchy level, the interface to the caller above is identical Redfish with no new protocol.
- **Unified Demultiplexing:** The demultiplexing mechanism remains identical at every step via UUID prefixes on every resource URI.
- **Seamless Expansion:** Adding a rack simply means registering a new Level-1 DM as an AggregationSource in the Level-2 DM.

HIERARCHICAL NAMESPACING

Level-1 DM:

```
/redfish/v1/Systems/{nodeUUID}.1
```

Level-2 DM:

```
/redfish/v1/Systems/{rackUUID}.{nodeUUID}.1
```

Level-3 DM:

```
/redfish/v1/Systems/{podUUID}.{rackUUID}.{nodeUUID}.1
```

WHY HIERARCHICAL DMs?

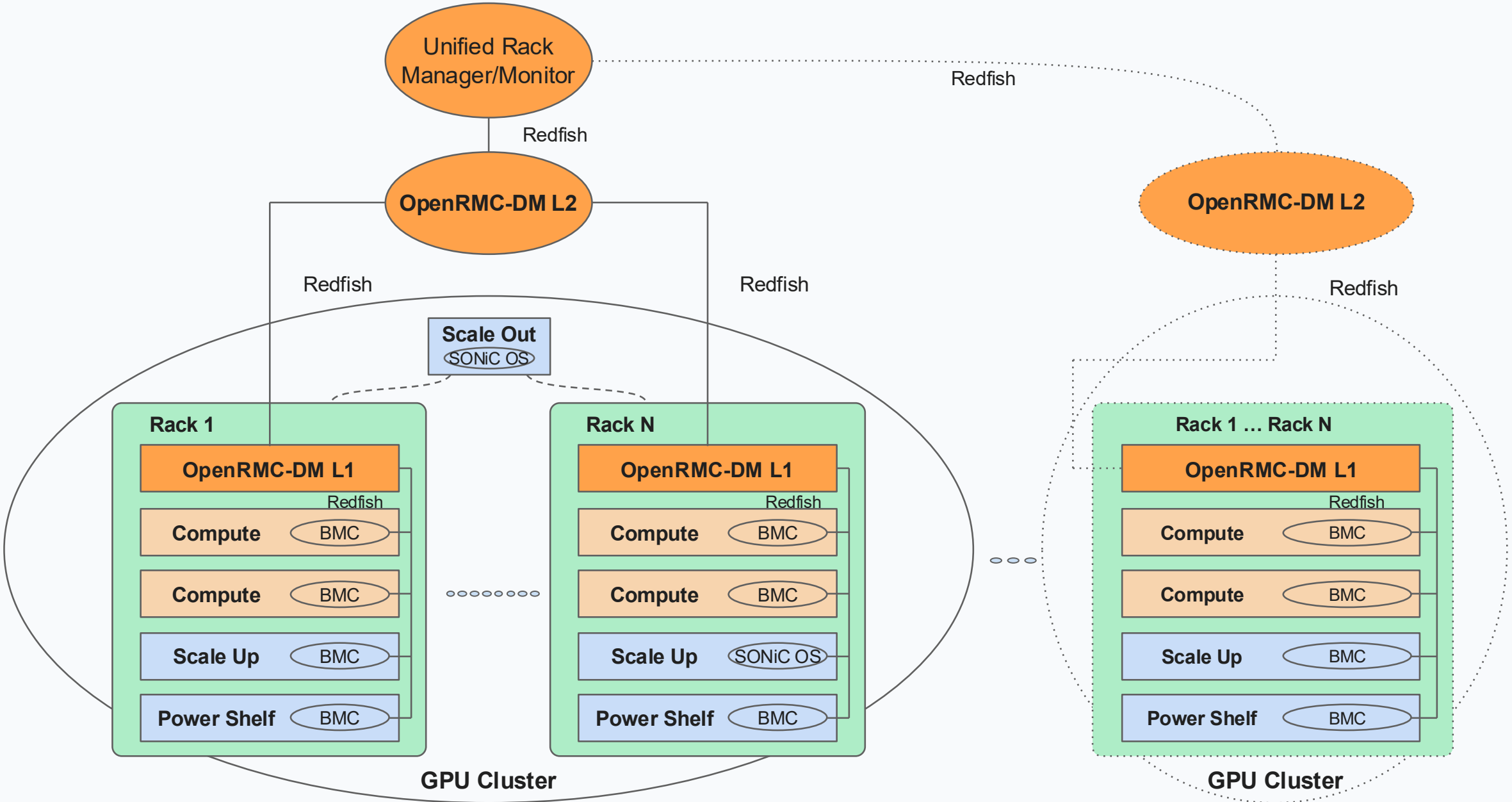
Without intermediate levels, a northbound system must own the complex fan-out, aggregation, and session management across hundreds of Level-1 DMs directly.

Hierarchical DMs absorb this complexity at the physical boundary, keeping the northbound application scope fixed.

Hierarchical DMs - A Sample Hierarchy



Hierarchical OpenRMC-DMs



Scenario: POST SimpleUpdate — Fan-Out Through Levels

Scenario setup: 1 pod, 10 racks, 30 BMCs per rack (300 systems total)

HIERARCHICAL TASK EXECUTION & FLOW

NorthBound Client & Level-3 DM (Data Center)

```
POST /redfish/v1/UpdateService/Actions/SimpleUpdate {"Targets":
["/redfish/v1/Chassis/{podUUID}"]}
```

Level-3 DM demultiplexes targets by **{podUUID}**, routes to Level-2 DM, and tracks 1 task completion.

Level-2 DM (Pod-A)

Demultiplexes by **{rackUUID}** and groups by rack. Fans out 10 parallel POSTs to Level-1 DMs and tracks 10 task completions.

Level-1 DMs (Rack-1 to Rack-10)

Each handles 30 parallel BMC sessions on dedicated rack management networks (full bandwidth, no contention), tracking 30 tasks.

KEY ADVANTAGES & ROBUSTNESS

Simultaneous Execution

All 10 racks execute updates simultaneously. The total time for 300 systems is kept roughly equivalent to a single rack's update time.

Bounded Complexity

Task tracking scales elegantly: Level-3 tracks 1 task; Level-2 tracks 10; Level-1 tracks 30. No single controller is overwhelmed.

Fault Isolation & Resilience

If Rack-3 fails mid-update, Racks 1, 2, and 4-10 complete successfully. Level-2 reports partial success up the chain to the client.

Boundary-Aware Intelligence — Built Into the Hierarchy

Each DM level applies intelligence using the physical context it owns, scoping capabilities to rack, pod, or DC.

INTELLIGENCE CAPABILITIES ACROSS THE HIERARCHY

- **Event Consolidation**
  Compresses N same-type events from child nodes into 1 clean boundary-level event. Avoids event storms.
- **Root Cause Inference**
  Analyzes multiple correlated event types locally to generate 1 synthesized root cause event up the chain.
- **Health Rollup**
  Evaluates active child health states to present a single consolidated boundary health summary.
- **Coordinated Operations**
  Handles complex tasks like fan-out, intelligent sequencing, and staggered recovery routines across children.
- **Resource Aggregation**
  Gathers capacity, hardware compliance, and aggregate power states scoped exactly to the boundary.

Result: Northbound client receives one unified health state, one root-cause event, and one consolidated view.

Call to Action

- Join the OpenRMC-DM sub-project.
  - OpenRMC-DM Subproject: https://www.opencompute.org/projects/openrmc-dm
- Participate in discussions to arrive at what could be the various levels of hierarchy for a Multi-Rack Management context (Rack/Pod/DC/Anything else that might make sense?).
- Contribute to standardization for nested URI namespaces.
- Contribute to a reference implementation of a 2-level prototype.

Thank You!

